

C-SW311: Software Design and Development Spring 2026

Submission and Discussion guidelines

Guidelines for project deliverables

1. For all diagrams, be **creative and rationale** on your assumptions about the information required, and try to include everything that is *important* for your model to be explanatory.
2. For EACH deliverable, each group must submit a well-written and organized **Technical Report document** containing ALL steps, tables and diagrams and also describing your solutions and rationales to the assignment (based on the deliverable requirements, preferably zipped or otherwise compressed). **Any assumptions you made during your work must be explicitly mentioned either in the Technical Report.**
3. Submit your **Technical Report document** in PDF format and No Handwriting will be accepted.
4. All submissions will be on Canvas and a submission link will be created for each deliverable.
5. **Plagiarism will be treated strictly.**
6. **NO LATE Submission will be accepted, and NO EXCEUSES**
7. **Any late submission will take ZERO.**
8. Please bear in mind that submission at the last minute might cause a network problem, and that would not be taken as an excuse. Therefore, you need to submit as early as possible on the submission day.
9. Discussions will be scheduled after the submission. Eng. Tawfik and Eng. Mohammed will return the submitted reports to each group with feedback highlighted inline (in the form of embedded comments).

Guidelines for Deliverable#1: Creational Design Patterns

Deadline of submission: Saturday 7th of March, 2025 at 11:59 pm.

1) Automated Java Code Generation

a) If you are continuing the same project from C-SW321

- If not already completed, use any **model transformation tool** to generate Java code from your **Design Class Diagram**.
- Your development environment must incorporate key **DevOps technologies**, including: **Eclipse or NetBeans, Git, Maven** and **JIRA**

b) If you have selected a new project idea

- First, create a **Design Class Diagram** for your new project.
- Use any **model transformation tool** to generate Java code from your diagram.
- Your development environment must incorporate key **DevOps technologies**, including: **Eclipse or NetBeans, Git, Maven** and **JIRA**

2) Creational Software Design Patterns

Based on your project, identify the following:

- a) **Two cases/problems** that require applying the **Singleton** design pattern.
- b) **One case/problem** that requires applying the **Factory Method** design pattern.
- c) **Two cases/problems** that require applying the **Abstract Factory** design pattern.

3) For Each Case/Problem Identified in Section (2)

- a) Develop a **UML Class Diagram** that conceptually illustrates your proposed solution.
- b) Implement your solution in **Java** within the project from Section (1).
 - You do **not** need to include code in the report.
 - However, your implementation must be **complete and running**, with **no errors**.

Deliverable Type

Students are required to submit the following:

a) Report

- A structured document explaining the identified cases/problems for each required creational design pattern.
- UML class diagrams for each case.
- Clear justification for why each pattern was selected and how it solves the identified problem.

b) NetBeans Project

- The complete and functional Java project.
- All implemented design patterns must be properly integrated.
- The project must compile and run without errors.
- The development environment should reflect the required DevOps tools (e.g., Maven configuration, Git repository usage, etc.).

Please ensure that both components are submitted before the deadline.